Linked List

Asprak TI UNESA ‘25

25 Februari, 2026

Linked List

Linked List (array pro-max :v) adalah struktur data linear yang terdiri dari kumpulan node.
Setiap node menyimpan:

- **Data**
- **Pointer (referensi)** ke node berikutnya

Berbeda dengan array, linked list **tidak disimpan bersebelahan di memori**, sehingga lebih fleksibel untuk penambahan dan penghapusan data (jadi kayak disimpen di suatu laci, nah setiap laci ada kode unik nya, jadi kalau mau ambil barang/hapus barang/nambahin barang itu tinggal referensi ke id laci nya)

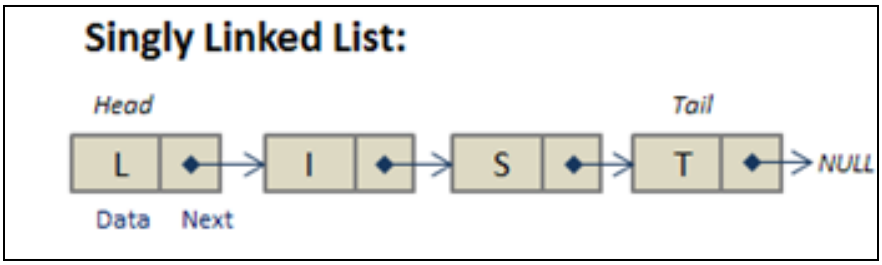
Linked List Characteristic

Single Linkedlist

Single Linked List adalah linked list yang setiap node-nya hanya memiliki:

- 1 data
- 1 pointer → menunjuk ke node berikutnya

Struktur:



Ciri-ciri:

Linkedlist (single, doubly, -> circular)

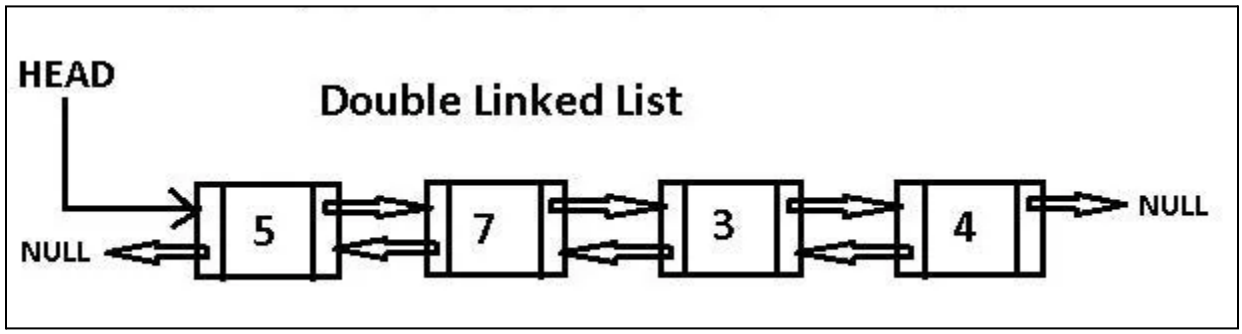
- Hanya bisa traversal **maju (forward)**
- Node terakhir menunjuk ke **NULL**
- Lebih sederhana dan hemat memori dibanding doubly

Doubly Linkedlist

Doubly Linked List adalah linked list yang setiap node-nya memiliki:

- 1 data
- 1 pointer ke node berikutnya (next)
- 1 pointer ke node sebelumnya (prev)

Struktur:



Ciri-ciri:

- Bisa traversal **maju dan mundur**
- Lebih fleksibel
- Butuh memori lebih besar (karena ada 2 pointer)

Cicular Linkedlist

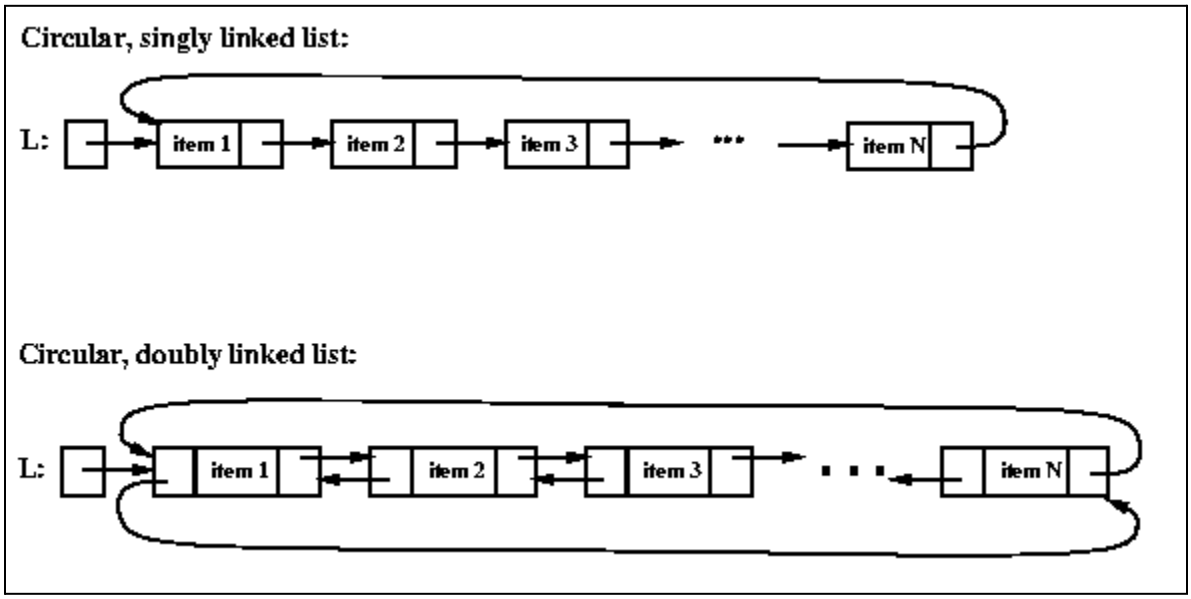
Linkedlist (single, doubly, -> circular)

Circular Linked List adalah linked list yang node terakhirnya **tidak menunjuk NULL**, tetapi kembali ke node pertama.

Bisa berbentuk:

- Circular Single Linked List
- Circular Doubly Linked List

Struktur :



Ciri-ciri:

- Tidak ada NULL
- Node terakhir menunjuk ke head
- Cocok untuk sistem yang berulang (misalnya round-robin scheduling)

Code Contoh

• **Deklarasi Node**

```
typedef struct TNode{
   int data;
   TNode *next;
};
```

• **Pembentukan Node Baru**

```
TNode *baru;
baru = new TNode;
baru->data = databaru;
baru->next = NULL;
```

- **Deklarasi Head**

```
TNode *head = NULL;
```

~ Circular Linked List (cara nyambung)

Linkedlist (single, doubly, -> circular)

Saat list kosong:

```
head = baru;
baru->next = head;
```

Insert depan (Circular):

```
TNode *bantu = head;

while(bantu->next != head){
   bantu = bantu->next;
}

baru->next = head;
bantu->next = baru;
head = baru;
```

KESIMPULAN

Linked List adalah struktur data linear yang terdiri dari sekumpulan node yang saling terhubung satu sama lain menggunakan pointer. Setiap node menyimpan dua komponen utama, yaitu data dan alamat (pointer) yang menunjuk ke node berikutnya. Berbeda dengan array yang disimpan secara berurutan di memori, linked list tidak membutuhkan lokasi memori yang bersebelahan. Node bisa berada di lokasi mana saja, lalu dihubungkan melalui pointer tersebut (yah kayak tadi, misal LINKEDLIST (node) = LACI ||| Nomer Laci = ID Node (pointer)... Nanti biar gampang mau ambil, ngerubah, atau menghapus dari laci dengan ID berapa) .

Karena tidak bergantung pada memori yang berurutan, ukuran linked list bersifat dinamis. Artinya, selama program berjalan, kita bisa menambah atau menghapus node tanpa perlu menentukan ukuran di awal. Inilah salah satu keunggulan utama linked list dibanding array. Pada array, jika kapasitas penuh, kita harus membuat array baru dengan ukuran lebih besar dan menyalin data. Pada linked list, kita cukup membuat node baru dan menyambungkannya ke struktur yang sudah ada.

Namun, cara mengakses data pada linked list berbeda dengan array. Jika pada array kita bisa langsung mengakses elemen menggunakan indeks (misalnya arr[3]), pada linked list kita harus memulai dari head (node pertama) lalu menelusuri satu per satu hingga menemukan node yang diinginkan. Proses ini disebut traversal dan bersifat sequential access. Karena itu, pencarian data

Linkedlist (single, doubly, -> circular)

pada linked list bisa lebih lambat dibanding array. (Nah ini ribet nanti di materi semisal SEARCHING, sasageyo dan semangat :v)

Tips and Tricck

Kalau dari aku, tips and trick nya sih, pahamin logikanya, kalau bingung bikin linkedlist nya penempatan node gimana, bisa digambar dulu di kertas.